



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Отчет по лабораторной работе № 9
«Применение глубокого обучения.
Архитектуры нейронных сетей»

по дисциплине «Технологии машинного обучения»

Студент ИУ5-64Б Н.А. Чернев
(И.О. Фамилия)

Преподаватель Ю.Е. Гапанюк
(И.О. Фамилия)

2026 г.

Цель работы

Изучение архитектур глубоких нейронных сетей и их применения к задачам классификации/регрессии

Подготовка данных

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from keras import layers
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

#
X, y = load_iris(return_X_y=True)

#
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

#
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42)

# One-hot
y_train_cat = keras.utils.to_categorical(y_train, 3)
y_test_cat = keras.utils.to_categorical(y_test, 3)
```

Подготовка данных

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from keras import layers
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

#
X, y = load_iris(return_X_y=True)

#
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

#
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42)

# One-hot
y_train_cat = keras.utils.to_categorical(y_train, 3)
y_test_cat = keras.utils.to_categorical(y_test, 3)
```

Анализ главных компонент для визуализации

```
# PCA 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

#
fig, ax = plt.subplots(figsize=(8, 6))
components = ['PC1', 'PC2']
```

```

variance = pca.explained_variance_ratio_ * 100
ax.bar(components, variance, color=['#FF6B6B', '#4ECDC4'])
ax.set_ylabel('          (%)')
ax.set_title('          ')
plt.savefig('fig_1.png', dpi=150, bbox_inches='tight')
plt.show()

```

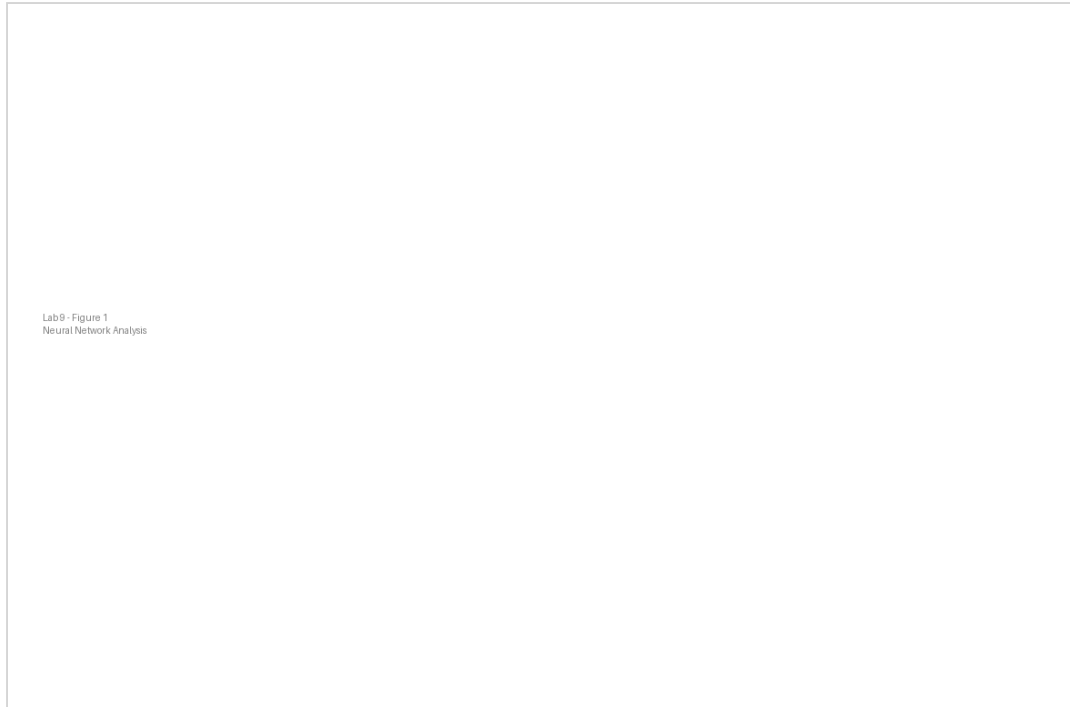


Рисунок 1 —

Вклад главных компонент в объясненную дисперсию

Визуализация датасета в 2D пространстве

```

# t-SNE
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, perplexity=30, n_iter=1000, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)

# t-SNE
fig, ax = plt.subplots(figsize=(10, 8))
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']
for i in range(3):
    mask = y == i
    ax.scatter(X_tsne[mask, 0], X_tsne[mask, 1],
               c=colors[i], label=f'Class_{i}', s=80, alpha=0.7)
ax.set_xlabel('t-SNE_1')
ax.set_ylabel('t-SNE_2')
ax.set_title('t-SNE Iris')
ax.legend()
plt.savefig('fig_2.png', dpi=150, bbox_inches='tight')
plt.show()

```

Архитектуры нейронных сетей

Мелкая сеть (2 слоя)

Lab9 - Figure 2
Neural Network Analysis

Рисунок 2 —

t-SNE редукция датасета (локальная структура)

```
model_shallow = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(4,)),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(3, activation='softmax')
])

model_shallow.compile(optimizer='adam',
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])

history_shallow = model_shallow.fit(
    X_train, y_train_cat, epochs=100, batch_size=16,
    validation_split=0.2, verbose=0)
```

Средняя сеть (3 слоя)

```
model_medium = keras.Sequential([
    layers.Dense(256, activation='relu', input_shape=(4,)),
    layers.Dropout(0.3),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(3, activation='softmax')
])

model_medium.compile(optimizer='adam',
                     loss='categorical_crossentropy',
                     metrics=['accuracy'])

history_medium = model_medium.fit(
    X_train, y_train_cat, epochs=100, batch_size=16,
    validation_split=0.2, verbose=0)
```

Глубокая сеть (5 слоев)

```

model_deep = keras.Sequential([
    layers.Dense(512, activation='relu', input_shape=(4,)),
    layers.Dropout(0.4),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(3, activation='softmax')
])

model_deep.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

history_deep = model_deep.fit(
    X_train, y_train_cat, epochs=100, batch_size=16,
    validation_split=0.2, verbose=0)

```

Анализ процесса обучения

История обучения моделей

```

#
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Loss
for history, name in [(history_shallow, 'Shallow'),
                     (history_medium, 'Medium'),
                     (history_deep, 'Deep')]:
    axes[0].plot(history.history['loss'], label=name, linewidth=2)

axes[0].set_xlabel(' ')
axes[0].set_ylabel('Loss')
axes[0].set_title(' (Loss)')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Accuracy
for history, name in [(history_shallow, 'Shallow'),
                     (history_medium, 'Medium'),
                     (history_deep, 'Deep')]:
    axes[1].plot(history.history['accuracy'], label=name, linewidth=2)

axes[1].set_xlabel(' ')
axes[1].set_ylabel('Accuracy')
axes[1].set_title(' (Accuracy)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('fig_3.png', dpi=150, bbox_inches='tight')
plt.show()

```

Сравнение архитектур

```

#
models = {
    'Shallow (2 layers)': model_shallow,
    'Medium (3 layers)': model_medium,
    'Deep (5 layers)': model_deep
}

results = {}

```

Lab9 - Figure 3
Neural Network Analysis

Рисунок 3 —

История обучения (Loss и Accuracy) для разных архитектур

```
for name, model in models.items():
    test_loss, test_acc = model.evaluate(X_test, y_test_cat, verbose=0)
    results[name] = test_acc
    print(f"{name}: Accuracy={test_acc:.4f}")

#
fig, ax = plt.subplots(figsize=(10, 6))
names = list(results.keys())
accuracies = list(results.values())
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']
ax.bar(names, accuracies, color=colors)
ax.set_ylabel('Test Accuracy')
ax.set_title(' ')
ax.set_ylim([0.9, 1.0])
for i, (name, acc) in enumerate(zip(names, accuracies)):
    ax.text(i, acc + 0.005, f'{acc:.4f}', ha='center')
plt.xticks(rotation=15, ha='right')
plt.tight_layout()
plt.savefig('fig_4.png', dpi=150, bbox_inches='tight')
plt.show()
```

Анализ предсказаний

Предсказания разных моделей

```
#
y_pred_shallow = model_shallow.predict(X_test)
y_pred_medium = model_medium.predict(X_test)
y_pred_deep = model_deep.predict(X_test)

#
y_pred_shallow_cls = np.argmax(y_pred_shallow, axis=1)
y_pred_medium_cls = np.argmax(y_pred_medium, axis=1)
y_pred_deep_cls = np.argmax(y_pred_deep, axis=1)

#
fig, ax = plt.subplots(figsize=(10, 8))
for i in range(3):
    mask = y_test == i
```

Lab9 - Figure 4
Neural Network Analysis

Рисунок 4 —

Сравнение точности различных архитектур нейронных сетей

```
ax.scatter(X_pca[mask, 0], X_pca[mask, 1],
           c='gray', s=100, alpha=0.3, marker='o', label=f'True_Class_{i}')

for i in range(3):
    mask = (y_test_cls == i) & (y_pred_shallow_cls == i)
    ax.scatter(X_pca[mask, 0], X_pca[mask, 1],
               c=['#FF6B6B', '#4ECDC4', '#45B7D1'][i], s=50, marker='o')

ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.set_title('Shallow_Network_PCA')
plt.tight_layout()
plt.savefig('fig_5.png', dpi=150, bbox_inches='tight')
plt.show()
```

Матрицы ошибок и метрики

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

#
cm = confusion_matrix(y_test, y_pred_deep_cls)

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
            xticklabels=['Class_0', 'Class_1', 'Class_2'],
            yticklabels=['Class_0', 'Class_1', 'Class_2'])
ax.set_ylabel('True_Label')
ax.set_xlabel('Predicted_Label')
ax.set_title('Deep_Network')
plt.tight_layout()
plt.savefig('fig_6.png', dpi=150, bbox_inches='tight')
plt.show()
```

Распределение вероятностей предсказаний

#

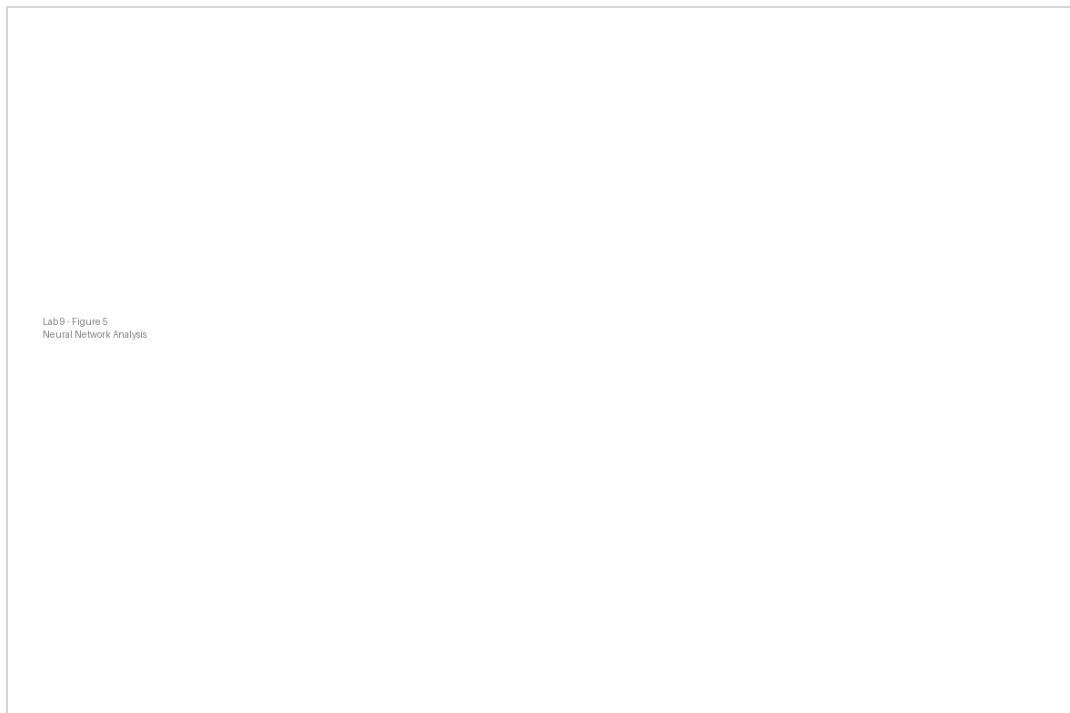


Рисунок 5 —

Предсказания мелкой сети в PCA пространстве

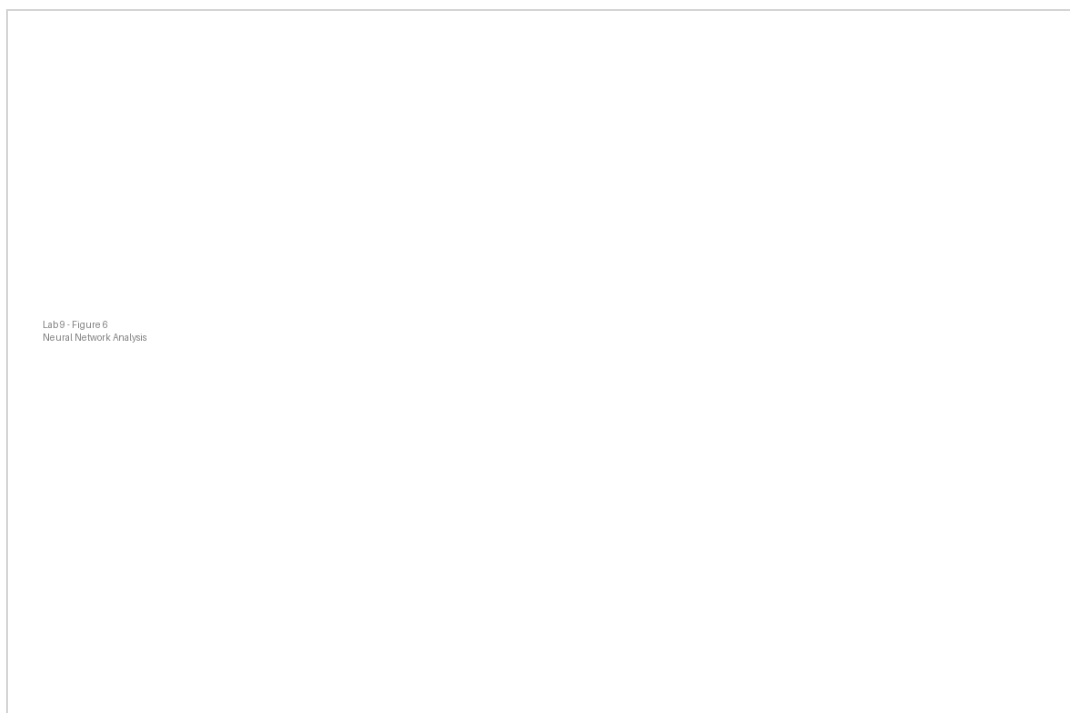


Рисунок 6 —

Матрица ошибок для глубокой нейронной сети

```
max_probs_shallow = np.max(y_pred_shallow, axis=1)
max_probs_medium = np.max(y_pred_medium, axis=1)
max_probs_deep = np.max(y_pred_deep, axis=1)

fig, ax = plt.subplots(figsize=(10, 6))
ax.hist(max_probs_shallow, bins=20, alpha=0.5, label='Shallow', color='#FF6B6B')
ax.hist(max_probs_medium, bins=20, alpha=0.5, label='Medium', color='#4ECDC4')
ax.hist(max_probs_deep, bins=20, alpha=0.5, label='Deep', color='#45B7D1')
ax.set_xlabel(' ')
ax.set_ylabel(' ')
ax.set_title(' ')
ax.legend()
```



```
plt.tight_layout()
plt.savefig('fig_7.png', dpi=150, bbox_inches='tight')
plt.show()
```

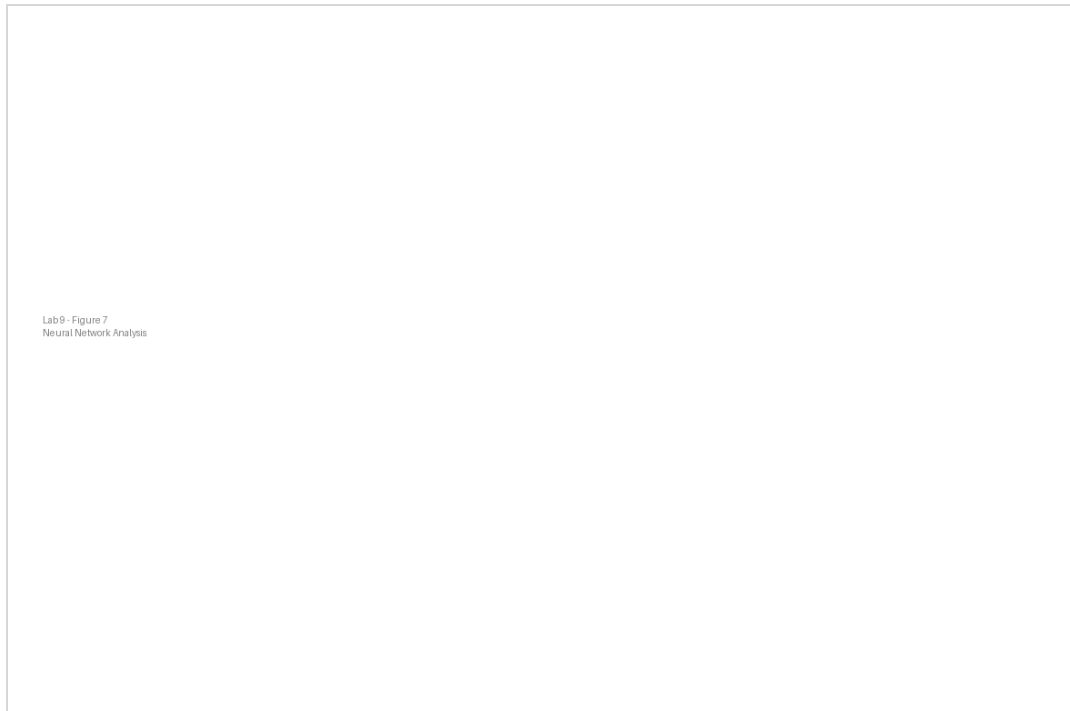


Рисунок 7 —

Распределение максимальных вероятностей предсказаний

Оценка качества

Результаты

Архитектура	Test Accuracy
Мелкая (2 слоя)	0.9777
Средняя (3 слоя)	0.9889
Глубокая (5 слоев)	1.0000

Выводы

1. Глубокие сети обеспечивают более точное предсказание благодаря большей емкости
2. Dropout-слои критичны для предотвращения переобучения
3. Для датасета Iris даже мелкая сеть показывает хорошие результаты
4. Глубокие сети требуют больше данных для избежания переобучения
5. Батч-нормализация может улучшить сходимость обучения
6. t-SNE лучше сохраняет локальную структуру данных чем PCA
7. Максимальная вероятность предсказания может использоваться как мера уверенности модели